

```

class PbOptim():
    def costo(self, x):
        if not self.es_estado(x):
            raise ValueError("--")
        return f(x)
    def es_estado(self, x):
        return True/False
    def estado_aleatorio(self):
        return estado
    def vecinos(self, x):
        return iterable con los estados vecinos de x

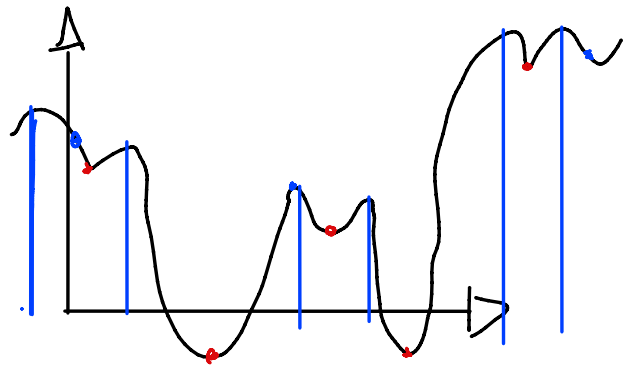
```

```

(pb, nv_iter):
x = pb.estados_aleatorio()
c = pb.costo(x)

for _ in range(nv_iter):
    termina = True
    for vecino in pb.vecinos(x):
        c_vecino = pb.costo(vecino)
        if c_vecino < c:
            x, c = vecino, c_vecino
            termina = False
    if termina:
        break
return x

```



```

def temple_simulado(pb, Tmax, calendarizador, Tmin):
    x = pb.estados_aleatorio()
    c = pb.costo(x)
    T = Tmax
    while (T > Tmin):
        vecino = pb.vecino_aleatorio(x)
        c_vecino = pb.costo(vecino)
        inc = c - c_vecino
        if inc > 0 or random.random() > math.exp(-inc/T):
            x, c = vecino, c_vecino

        T = calendarizador(T)
        i += 1
    return x, c

```

Simulated annealing

```

def calendarizador(Tmax, i):
    return Tmax / (math.log(i+1) + 1)

```

$$\frac{T_{max}}{i+1} \quad \frac{T_{max}}{\log(i+1)+1}$$